Department of Mathematics
and Computer Science

University of Southern Denmark
Bachelor Thesis in Artificial Intelligence

Approximate Similarity Search:

Evaluation on Image Retrieval, Out-of-Distribution Detection, and Recommender Systems

Author(s)
Emil Mygind Holm
emmyg21@student.sdu.dk

Supervisor
Ricardo J. G. B. Campello
Professor

May 2026

SDU

Abstract

English Approximate nearest neighbor (ANN) search enables efficient similarity retrieval in high-dimensional spaces by trading a small loss in accuracy for substantial gains in query speed. This thesis evaluates the practical impact of ANN approximation across three downstream tasks: image-based similarity retrieval, out-of-distribution (OoD) detection, and item-based collaborative filtering recommendation. Two ANN algorithms, Locality Sensitive Hashing (LSH) and Hierarchical Navigable Small World graphs (HNSW), are compared against exact k-nearest neighbor search using ImageNet100 feature vectors and the MovieLens 10M dataset. The results demonstrate that HNSW consistently provides a more favorable speed-accuracy trade-off than LSH across all evaluated settings, achieving recall values close to exact search while reducing query time by approximately a factor of five. For OoD detection, HNSW preserved AUC-ROC performance for both evaluated scoring functions, though an unexpected sensitivity ordering was observed: DNN proved more sensitive to approximation error than NN-Guide, suggesting that scoring functions aggregating over multiple neighbours are inherently more robust than those relying on a single nearest-neighbour statistic. For recommendation, HNSW matched exact search quality at moderate index parameters while substantially reducing query time. Taken together, the findings suggest that the practical impact of ANN approximation depends primarily on the sensitivity characteristics of the downstream task rather than on nearest-neighbour recall alone.

Danish Approximate nearest neighbor (ANN) search muliggør effektiv similarity-søgning i høj-dimensionelle rum ved at udveksle et lille præcisionstab mod betydelige gevinster i query-hastighed. Denne afhandling evaluerer den praktiske effekt af ANN approximation på tværs af tre anvendelsesopgaver: billedbaseret similarity retrieval, out-of-distribution (OoD) detection og item-based collaborative filtering til anbefalingssystemer. To ANN-algoritmer, Locality Sensitive Hashing (LSH) og Hierarchical Navigable Small World graphs (HNSW), sammenlignes med eksakt k-nearest neighbor search ved brug af ImageNet100 feature-vektorer og MovieLens 10M-datasættet. Resultaterne viser, at HNSW konsekvent giver et mere fordelagtigt speed-accuracy trade-off end LSH på tværs af alle evaluerede indstillinger, med recall-værdier tæt på eksakt søgning og en reduktion i query-tid på approximately en faktor fem. For OoD detection bevarede HNSW AUC-ROC-

ydelsen for begge evaluerede scoring functions, selvom en uventet sensitivity ordering blev observeret: DNN viste sig mere følsom over for approximation error end NN-Guide, hvilket tyder på, at scoring functions der aggregerer over flere naboer er mere robuste end metoder der baserer sig på en enkelt nearest neighbor-statistik. For anbefalingssystemer matchede HNSW eksakt søgning ved moderate indeksparametre og reducerede query-tid betydeligt. Samlet set indikerer resultaterne, at den praktiske effekt af ANN approximation primært afhænger af downstream-opgavens sensitivity characteristics snarere end af nearest neighbor recall alene.

Use of AI

Generative AI tools were used as supporting tools during the development of this thesis. Their use included assistance with report structuring, grammar and language correction, plotting and formatting of figures, code suggestions and debugging assistance, as well as rephrasing and improving sentence formulations for readability and academic tone.
All experimental design, implementation, analysis, interpretation of results, and final conclusions were carried out and critically evaluated by the author. AI-generated suggestions were manually reviewed, validated, and modified where necessary before inclusion in the final report.

Contents

1 Introduction **3**

2 Research Question(s) and Hypothesis(es) **5**

3 Methods and Materials **7**

3.1 Datasets 7

3.1.1 ImageNet100 7

3.1.2 MovieLens 10M 8

3.2 Algorithms 8

3.2.1 Exact k-Nearest Neighbors 8

3.2.2 Locality Sensitive Hashing (LSH) 9

3.2.3 Hierarchical Navigable Small World (HNSW) 9

3.3 Out-of-Distribution Detection Methods 10

3.3.1 Deep Nearest Neighbors 10

3.3.2 NN-Guide 10

3.4 Recommendation System 11

3.4.1 Item-Based k-Nearest Neighbors 11

4 Results and Analyses **12**

4.1 Similarity Search Task 12

4.2 Out-of-Distribution Detection Task 19

4.3 Recommendation Task 22

5 Discussion **24**

5.1 Similarity Search: HNSW as a Near-Lossless Approximation 24

5.2 Out-of-Distribution Detection: An Unexpected Sensitivity Ordering 25

5.3 Recommendation: Collaborative Filtering is Robust to Approximation . . . 25

5.4 Cross-Task Synthesis 26

6 Conclusions and Perspectives **27**

Appendix: Individual Report **31**

Chapter 1

Introduction

When a streaming service suggests the next movie to watch, or a music app assembles a personalised playlist, the underlying mechanism is often the same: finding items in a large dataset that are most similar to what the user already likes. This action is known as nearest neighbor search, and it appears in a wide range of machine learning applications, from image search and fraud detection to medical diagnosis and content recommendation.

In its basic form, k-nearest neighbor (kNN) search works by comparing a query point against every item in the database and returning the k closest ones. This is reliable, but slow. As datasets grow larger, comparing every query against millions of stored vectors becomes a bottleneck that makes real-time use impractical.

Approximate nearest neighbor (ANN) search solves this by accepting a small chance of returning imperfect results in exchange for much faster query times. Two well-known algorithms are Locality Sensitive Hashing (LSH), which groups similar items into the same hash bucket, and Hierarchical Navigable Small World graphs (HNSW), which builds a layered graph structure that allows the search to navigate quickly towards the most relevant items.

The key question this thesis investigates is not simply whether ANN search is faster than exact search, but whether the small errors it introduces matter in practice. This depends heavily on what the search results are used for. In some tasks, a slightly imperfect neighbor set may have no noticeable effect on the final outcome. In others, it could be more costly.

To explore this, three application tasks are evaluated: image retrieval, out-of-distribution (OoD) detection, and movie recommendation. These three tasks were chosen because they represent different levels of sensitivity to approximation errors. Image retrieval measures search quality directly. OoD detection uses nearest neighbors to identify inputs that fall outside the training distribution, where accuracy matters for reliability. Recommender systems are naturally tolerant of small imprecisions, since there is rarely one single correct answer.

This thesis compares exact kNN, LSH, and HNSW across all three tasks through experiments on ImageNet100 image features and the MovieLens 10M dataset. The goal is to understand under which conditions approximate search is a safe replacement for exact search, and where the trade-off between speed and accuracy becomes a concern.

Chapter 2

Research Question(s) and Hypothesis(es)

Approximate nearest neighbor search introduces a fundamental trade-off: queries can be answered faster, but at the cost of occasionally returning imperfect results. Whether this cost is acceptable depends entirely on the downstream task. This chapter formalizes the questions that this thesis investigates and states the corresponding hypotheses.

RQ1 Speed-Accuracy Trade-off in Similarity Search
To what extent do approximate nearest neighbor algorithms (LSH and HNSW) reduce query time compared to exact k-nearest neighbor search, and how much recall is sacrificed?
Exact nearest neighbor search guarantees perfect recall but scales poorly with dataset size, as every query requires a full comparison against the database. LSH and HNSW offer sub-linear query times by trading exactness for speed. The central question is whether the achieved speedup is substantial enough to justify the loss in recall, and how the two algorithms compare against each other across their respective hyperparameter settings.
H1: We hypothesize that both LSH and HNSW will achieve significantly lower amortized query times than exact kNN. Based on the known properties of HNSW [6] and LSH [4], and supported by comparative benchmarks [1], we expect HNSW to reach near-perfect recall across a wider range of configurations, while LSH will require large bit-widths to approach comparable recall levels, resulting in a less favorable speed-accuracy trade-off overall.

RQ2 Approximate Search in Out-of-Distribution Detection
Can approximate nearest neighbor search maintain competitive AUC-ROC performance in OoD detection methods (Deep Nearest Neighbors and NN-Guide) relative to exact kNN?
OoD detection methods based on nearest neighbors derive their scores from the distances

or similarities to training-set neighbors. If the approximate index returns incorrect neighbors, the resulting scores may be unreliable, and the detector's ability to separate in-distribution from out-of-distribution samples may decline. This question asks whether that reduction is meaningful in practice.
H2: We hypothesize that HNSW with sufficiently large efSearch will retain AUC-ROC values close to exact kNN for both DNN [9] and NN-Guide [7]. At low efSearch, we expect a measurable drop, particularly for NN-Guide, which directly uses the similarity value of the nearest neighbor in its scoring function and may therefore be more sensitive to approximation errors than DNN.

RQ3 Approximate Search in Recommender Systems
Does the reduced recall of approximate nearest neighbor search cause a measurable degradation in recommendation quality (Hit Rate@k), and is the speed-quality trade-off acceptable for an item-based collaborative filtering system?
In an item-based collaborative filtering setting, the recommender retrieves the k most similar items to a user's rated movies. If the approximate index misses relevant neighbors, the final recommendation list may change. The question is whether this change translates into a practically significant drop in Hit Rate, or whether the recommendation quality is robust to the approximation error.
H3: We hypothesize that the Hit Rate@k of HNSW-based item-based kNN will remain close to exact kNN for moderate efSearch values, while achieving substantially lower query times. Because recommendation is inherently tolerant of small imprecisions [8], we expect the quality loss to be small relative to the speedup gained, making the trade-off acceptable in practice.

Chapter 3

Methods and Materials

3.1 Datasets

Two data sources are used across the three experimental tasks in this thesis. Image data from ImageNet100 and Places365 are used for the similarity search and out-of-distribution detection experiments, while the MovieLens 10M dataset is used for the recommender system experiment. The following sections describe each in turn.

3.1.1 ImageNet100

ImageNet100 is a selected subset of the full ImageNet collection [2], containing images from 100 object categories such as cars and chairs. For the purposes of this project, the collection is pre-organized into two subsets: InDSearch, containing approximately 95,000 images, and InDQuery, containing approximately 40,000 images, each covering the same 100 classes.

All images are resized to 224×224 pixels and passed through a WideResNet-50-2 neural network [10] to extract 2,048-dimensional feature vectors. Two network configurations are used: Conf1, which uses standard ImageNet-pretrained weights, and Conf2, which uses contrastively fine-tuned weights. The motivation for the two configurations is tied to the OoD detection methods described in Section 3.3. Feature extraction is computationally demanding, particularly for Conf2 which uses a fine-tuned network. To avoid repeated computation, all feature vectors were extracted once and stored on disk prior to the experiments.

In the similarity search task, InDSearch serves as the database being searched, while InDQuery provides the query vectors. In the OoD detection task, InDSearch serves as the in-distribution reference set, InDQuery as in-distribution test queries, and Places365 as out-of-distribution test queries.

Places365 [11] is a collection of 36,500 scene images across 365 indoor and outdoor categories such as kitchens, streets, and offices. Since it depicts a fundamentally different

type of visual content than ImageNet100, it serves as the out-of-distribution query set in the OoD experiments. The same preprocessing is applied: images are resized to 224×224 pixels and feature vectors are extracted using both Conf1 and Conf2.

3.1.2 MovieLens 10M

The MovieLens 10M dataset [5] contains 10,000,000 ratings of 10,677 movies by 69,878 users, on a scale from 0.5 to 5 in half-point increments. Each rating is accompanied by a timestamp, which is used in the recommender system evaluation.
Each movie is represented as a sparse vector of length 69,878, where each dimension corresponds to a user and the value at that position is the rating that user gave the movie, or zero if no rating exists. Since most users have only rated a small fraction of all movies, these vectors are highly sparse. For the similarity search experiments, cosine distance is used as the similarity measure, as it handles sparse vectors well by focusing on the pattern of interactions rather than their absolute magnitude.
For the recommender system task, the dataset is split into training and test sets using a timestamp-based strategy: the last movie each user rated is held out as the test item, and the remaining ratings are used for training. This reflects a realistic scenario where the goal is to predict the next item a user will interact with.

3.2 Algorithms

3.2.1 Exact k-Nearest Neighbors

Exact k-nearest neighbor (kNN) search is the baseline method against which all approximate algorithms are evaluated. Given a query vector $\mathbf{q}$ and a database of n vectors, exact kNN retrieves the k vectors with the smallest distance to $\mathbf{q}$ by comparing it against every vector in the database. This guarantees that the true nearest neighbors are always returned, but at a computational cost that grows linearly with n.
There are two distance measures used in this project. The Euclidean distance between vectors $\mathbf{x}$ and $\mathbf{y}$ is defined as:

$$d_{L2}(\mathbf{x}, \mathbf{y}) = |\mathbf{x} - \mathbf{y}|2 \tag{3.1}$$

The cosine distance is defined as:

$$d\cos(\mathbf{x}, \mathbf{y}) = 1 - \frac{\langle \mathbf{x}, \mathbf{y} \rangle}{|\mathbf{x}|_2 |\mathbf{y}|_2} \tag{3.2}$$

When vectors are L2-normalized beforehand, cosine distance reduces to inner product similarity, which is exploited by several of the algorithms in this project. Exact kNN is

implemented using FAISS [3] via `IndexFlatL2` and `IndexFlatIP` respectively.

3.2.2 Locality Sensitive Hashing (LSH)

Locality Sensitive Hashing [4] is an approximate nearest neighbor algorithm that maps high-dimensional vectors to compact binary hash codes, such that similar vectors are likely to produce the same or similar hash codes. The key idea is that instead of comparing a query against every vector in the database, the search is restricted to vectors that share the same hash bucket, which drastically reduces the number of comparisons needed.
In the FAISS implementation used here (`IndexLSH`), each vector is projected onto `nbits` random hyperplanes. For each hyperplane, the projection produces either a 0 or a 1 depending on which side of the hyperplane the vector falls on, resulting in a binary code of length `nbits`. Similarity is then approximated by the Hamming distance between hash codes, which is the number of bit positions where two codes differ.
The single hyperparameter controlling the trade-off between speed and accuracy is `nbits`: a larger value produces longer, more discriminative codes that better preserve the original similarity structure, but increases both memory usage and comparison time. A smaller value is faster but loses more information, leading to lower recall. In this project, `nbits` $\in 64, 128, 256, 512, 1024, 2048, 4096, 8192$ is evaluated for image data and `nbits` $\in 256, 512, 1024, 2048, 3000, 4096, 6144$ for MovieLens.
Since FAISS's LSH implementation uses inner product internally, all vectors are normalized to unit length before indexing, which effectively makes the search equivalent to cosine similarity.

3.2.3 Hierarchical Navigable Small World (HNSW)

HNSW [6] is an approximate nearest neighbor algorithm that organizes data points into a layered proximity graph, where each node represents a vector and edges connect nearby vectors. The intuition is borrowed from the concept of small world networks: in a well-connected graph, any two nodes can be reached from each other in a small number of steps. HNSW exploits this by building multiple layers of graphs with decreasing density, allowing the search to start at a coarse level and progressively zoom in towards the nearest neighbors.
At query time, the algorithm starts at the top layer with few connections and navigates greedily towards the query vector, moving to the next layer down once a local minimum is reached. This hierarchical traversal means that most of the database is never visited, resulting in query times that are significantly faster than exact search.
HNSW has three main hyperparameters.

M defines the maximum number of edges per node in the graph a higher value improves recall but increases memory usage and index build time. `efConstruction` controls how many candidate neighbors are considered when building the graph larger values produce a higher quality graph at the cost of longer build times. `efSearch` controls how many nodes are tracked during a query increasing this value explores more of the graph and improves recall, but also increases query time. In this project, $M \in 16, 32, 256$, `efConstruction` $\in 16, 64, 256$, and `efSearch` $\in 32, 256, 512, 1024$ are evaluated for image data, and $M \in 8, 16, 32$, `efConstruction` $\in 8, 16, 32$, `efSearch` $\in 4, 8, 16, 32, 64$ for MovieLens. HNSW is implemented via `IndexHNSWFlat` in FAISS [3].

3.3 Out-of-Distribution Detection Methods

3.3.1 Deep Nearest Neighbors

Deep Nearest Neighbors (DNN) [9] is a simple OoD detection method that uses the distance from a query vector to its nearest neighbors in the training set as an anomaly score. The intuition is that in-distribution samples tend to cluster tightly in feature space, while out-of-distribution samples tend to land further away from any training point.
Given a query vector $\mathbf{q}$, the in-distribution score is defined as:

$$S_{InD}(\mathbf{q}) = -d_{L2,k}(\mathbf{q}) \tag{3.3}$$

where $d_{L2,k}(\mathbf{q})$ is the Euclidean distance from $\mathbf{q}$ to its k-th nearest neighbor in the reference set InDSearch. A high score indicates that the query is close to known training points and is therefore likely in-distribution, while a low score suggests the query is far from any training point and may be out-of-distribution.
DNN is used with feature vectors extracted using Conf2, the contrastively fine-tuned WideResNet-50-2, as contrastive training is known to produce more compact and well-separated clusters in feature space, which makes distance-based scoring more reliable.

3.3.2 NN-Guide

NN-Guide [7] is a more sophisticated OoD detection method that combines classification confidence with nearest neighbor similarity to produce an in-distribution score. The intuition is that an in-distribution sample should both be classified with high confidence and be similar to its nearest neighbors in the training set. If either condition fails, the sample is more likely to be out-of-distribution.

Given a query vector $\mathbf{q}$, the in-distribution score is defined as:

$$S_{InD}(\mathbf{q}) = \text{conf}(\mathbf{q}) \cdot \text{sim}(\mathbf{q}) \tag{3.4}$$

where $\text{conf}(\mathbf{q})$ is a confidence score derived from the classification output of the neural network, and $\text{sim}(\mathbf{q})$ is the confidence-weighted average cosine similarity between $\mathbf{q}$ and its k nearest neighbors in InDSearch. Two confidence measures are evaluated in this project: Maximum Softmax Probability (MSP) and Negative Energy.
NN-Guide is used with feature vectors extracted using Conf1, the standard ImageNet-pretrained WideResNet-50-2. Unlike DNN, the scoring function directly incorporates the similarity value of the nearest neighbors, which means it may be more sensitive to approximation errors when exact search is replaced by an ANN algorithm.

3.4 Recommendation System

3.4.1 Item-Based k-Nearest Neighbors

The recommendation task is formulated as next-item prediction using item-based collaborative filtering [8]. Given a user's watch history, the system retrieves the k most similar movies to the last item the user watched, and recommends those as candidates for the next movie. A recommendation is considered a hit if the user's actual next movie appears within the top-k recommendations.
Data representation. Each movie is represented as a 69,878-dimensional vector where each element corresponds to a user's rating of that movie, or zero if no rating exists. Ratings are kept as continuous values rather than binarized, and all vectors are normalized to unit length prior to indexing. Cosine similarity is used as the similarity measure, which reduces to inner product after normalization.
Train/test split. The dataset is split using a timestamp-based strategy: for each user, the last rated movie is held out as the test item, and all remaining ratings are used to build the item index. This setup reflects a realistic scenario where past interactions are used to predict the next interaction.
Evaluation. Recommendation quality is measured using Hit Rate@k, defined as the fraction of users for whom the held-out movie appears within the top-k recommendations. Experiments are conducted across $k \in 1, 5, 10, 30$ and evaluated on 50
ANN configurations. The same FAISS implementations of exact kNN, LSH, and HNSW used in the similarity search task are applied here, allowing a direct comparison between search quality (Recall@k) and recommendation quality (Hit Rate@k) across all algorithm configurations.

Chapter 4

Results and Analyses

4.1 Similarity Search Task

This section presents the results of the similarity search experiments conducted on ImageNet100 and MovieLens 10M. We evaluate exact kNN, LSH, and HNSW across different hyperparameter configurations, measuring Recall@k and query time to assess the trade-off between search quality and computational efficiency.

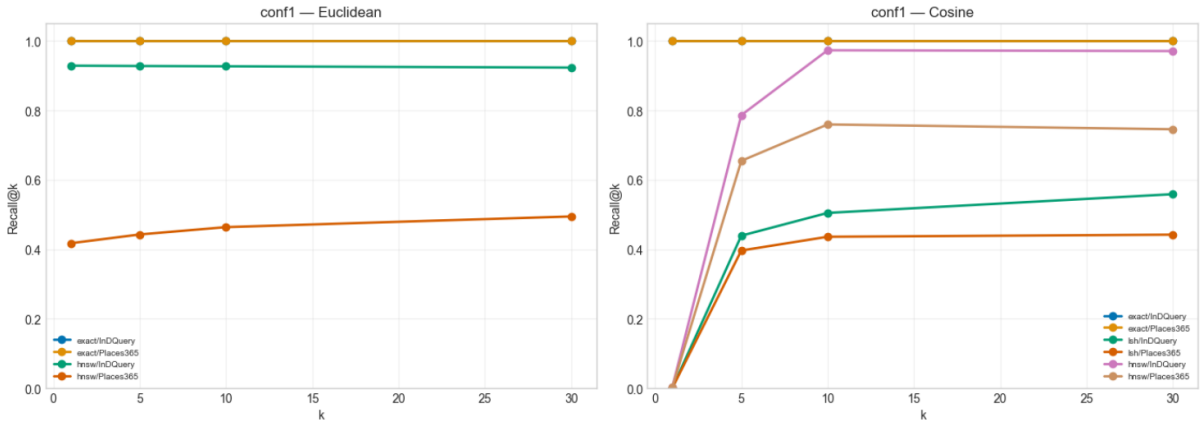


Figure 4.1: Recall@k for Conf1 (Euclidean and Cosine) across exact kNN, HNSW, and LSH for InDQuery and Places365.

Figure 4.1 shows Recall@k for $k \in 1, 5, 10, 30$ across both neural network configurations and distance measures. Exact kNN achieves perfect recall of 1.0 across all settings, as expected. HNSW and LSH both show varying levels of recall depending on the configuration.

For Conf1 with Euclidean distance, HNSW achieves a stable recall of approximately 0.93 for InDQuery across all values of k, while Places365 queries reach only around 0.42. One possible explanation is that out-of-distribution images occupy sparser regions of the feature space, making it harder for HNSW to navigate towards their true nearest neighbours

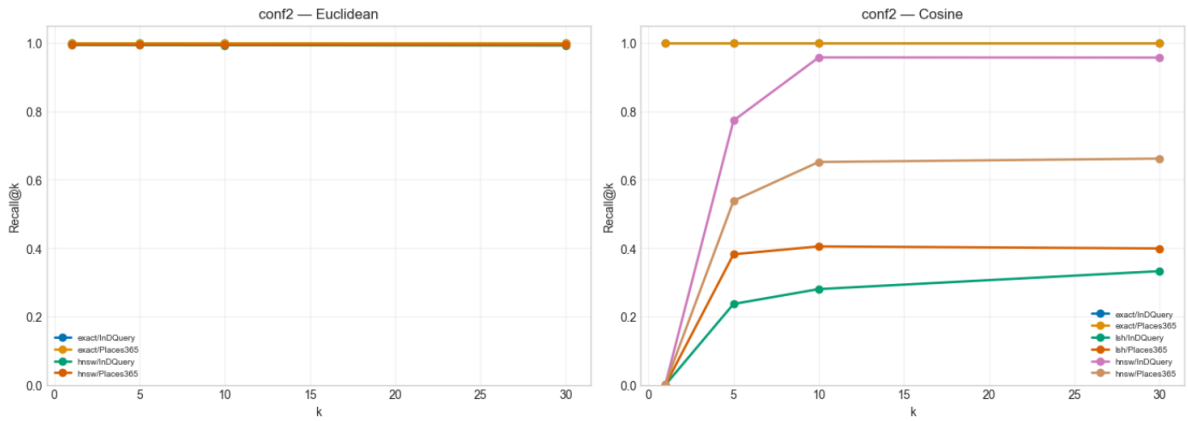


Figure 4.2: Recall@k for Conf2 (Euclidean and Cosine) across exact kNN, HNSW, and LSH for InDQuery and Places365.

using Euclidean distance.

Switching to Conf2 with Euclidean distance, both InDQuery and Places365 reach near-perfect recall close to 1.0. This is consistent with the properties of contrastive training, which produces more compact and well-separated clusters in feature space, making Euclidean distances more reliable as a proximity measure.

With Conf1 and cosine distance, HNSW improves noticeably for InDQuery, reaching approximately 0.98, while Places365 also improves compared to the Euclidean case. The normalization inherent in cosine similarity appears to compensate for the less structured feature space of Conf1. LSH is also present in the cosine plots but reaches substantially lower recall, peaking around 0.75 for Places365 at $k = 30$ even at its best configuration. Conf2 with cosine distance shows a similar pattern, though LSH performs slightly worse here than with Conf1, suggesting that the contrastively fine-tuned feature space is harder for LSH to approximate via random projections.

Overall, the choice of distance measure has a larger effect on HNSW recall than the choice of neural network configuration, while LSH consistently underperforms across all combinations.

Overall, the results suggest that retrieval quality depends strongly on the interaction between the distance measure and the underlying feature representation, while LSH consistently underperforms across all evaluated combinations

Having established the recall characteristics of each algorithm, we now turn to the computational side of the trade-off by examining index build time, average query time, and amortized time per query

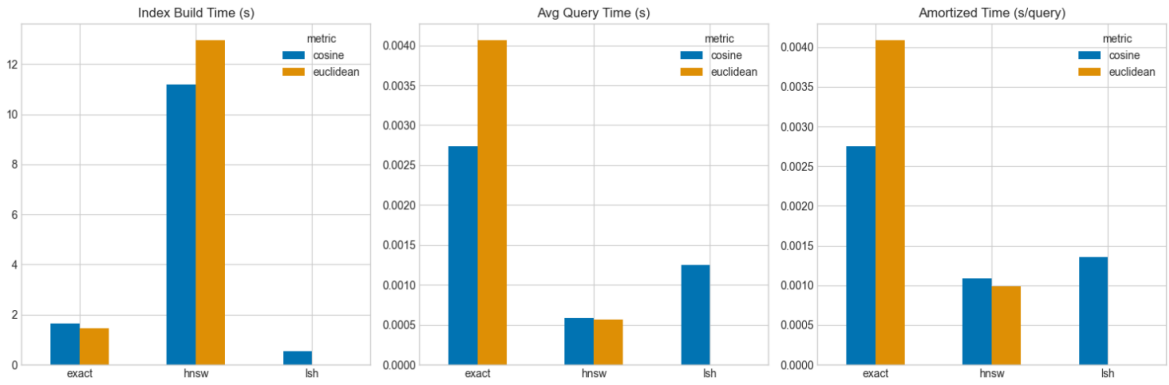


Figure 4.3: Index build time, average query time, and amortized time per query for exact kNN, HNSW, and LSH.

Figure 4.3 shows index build time, average query time, and amortized time per query for each algorithm across cosine and Euclidean distance measures.

In terms of index build time, LSH is by far the fastest to build, taking under one second, while exact kNN takes approximately 1.5 seconds. HNSW requires significantly more time to build, around 11 seconds for cosine and 13 seconds for Euclidean, reflecting the cost of constructing the proximity graph. Despite this overhead, HNSW compensates through substantially lower query times.

For average query time, exact kNN with Euclidean distance is notably slower than with cosine, taking approximately 0.004ms versus 0.0027ms. One likely explanation is that cosine similarity with normalized vectors reduces to inner product, which FAISS is highly optimized for, whereas Euclidean distance requires additional computation per comparison. HNSW shows similar query times for both distance measures, and both are considerably faster than exact kNN. LSH falls between the two in query time but with much lower recall as seen in the previous section.

The amortized time, which spreads the build cost across all queries, shows a similar pattern. HNSW remains competitive once the build cost is distributed, and LSH's advantage from its fast build time narrows when query time is also taken into account. Note that LSH only appears with cosine distance, as the FAISS implementation of LSH relies on inner product projections internally and does not support Euclidean distance directly.

With both recall and timing results in hand, we can now combine these into a direct visualization of the speed-accuracy trade-off.

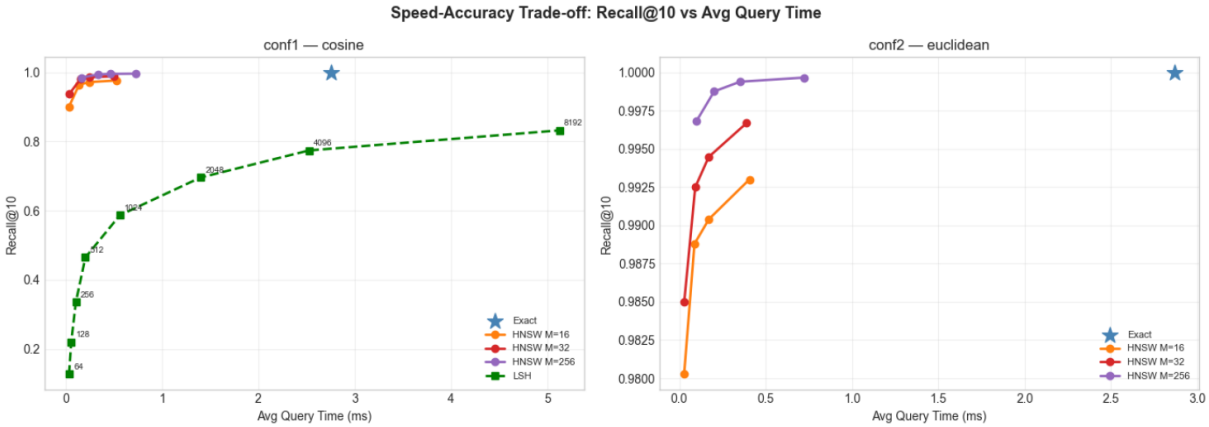


Figure 4.4: Speed-accuracy trade-off for Conf1 (cosine) and Conf2 (Euclidean): Recall@10 vs average query time.

Figure 4.4 shows Recall@10 plotted against average query time for Conf1 with cosine distance and Conf2 with Euclidean distance. Each HNSW curve represents a fixed value of M with efSearch increasing along the curve, while the LSH curve moves right as nbits increases.

For Conf1 with cosine distance, HNSW clearly dominates the trade-off frontier. All three M values achieve recall above 0.9 at query times well below exact kNN, with $M = 256$ reaching near-perfect recall at approximately 0.5ms compared to exact kNN at 2.7ms. This represents a speedup of roughly 5x while maintaining recall above 0.99. LSH on the other hand shows a much less favorable curve, requiring query times approaching 4ms to reach only 0.83 recall at nbits=8192, making it both slower and less accurate than HNSW in most configurations.

For Conf2 with Euclidean distance, the y-axis is compressed between 0.98 and 1.0, reflecting that HNSW already achieves very high recall across all configurations in this setting. The pattern is otherwise consistent: HNSW achieves near-exact recall at a fraction of the query time, and larger M values push the curve further towards the top-left corner of the plot.

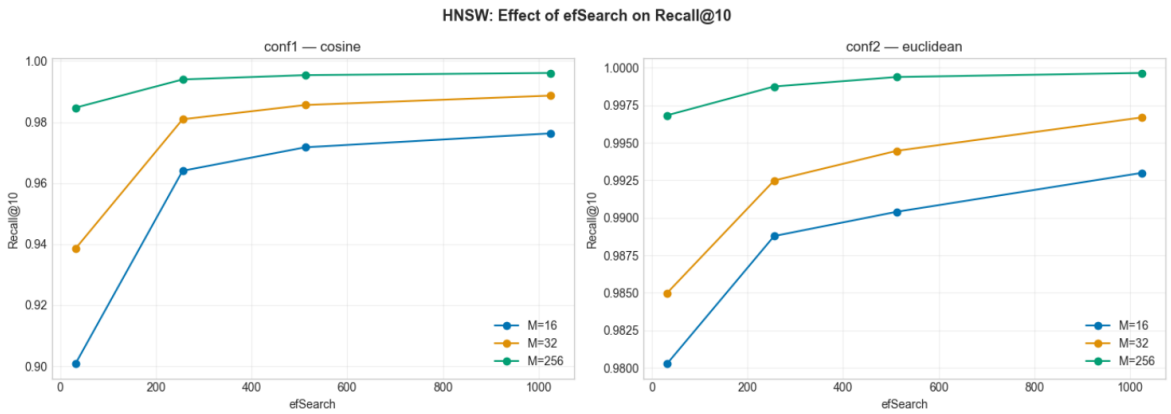


Figure 4.5: Effect of efSearch on Recall@10 for HNSW across different values of M, for Conf1 (cosine) and Conf2 (Euclidean).

Figure 4.5 shows the effect of efSearch on Recall@10 for $M \in 16, 32, 256$ across both Conf1 cosine and Conf2 Euclidean. In both cases, recall increases consistently with efSearch, but with diminishing returns at higher values. The effect of M is equally clear: higher M produces better recall at every efSearch value, with $M = 256$ starting above 0.985 even at the lowest efSearch setting for Conf2 Euclidean. For Conf1 cosine, $M = 16$ starts at approximately 0.90 at efSearch=32 and reaches only 0.975 at efSearch=1024, while $M = 256$ starts at 0.985 and plateaus around 0.996. This suggests that increasing efSearch alone cannot fully compensate for smaller M values.

With both recall and timing results in hand, we now examine how the individual hyperparameters of HNSW affect search quality.

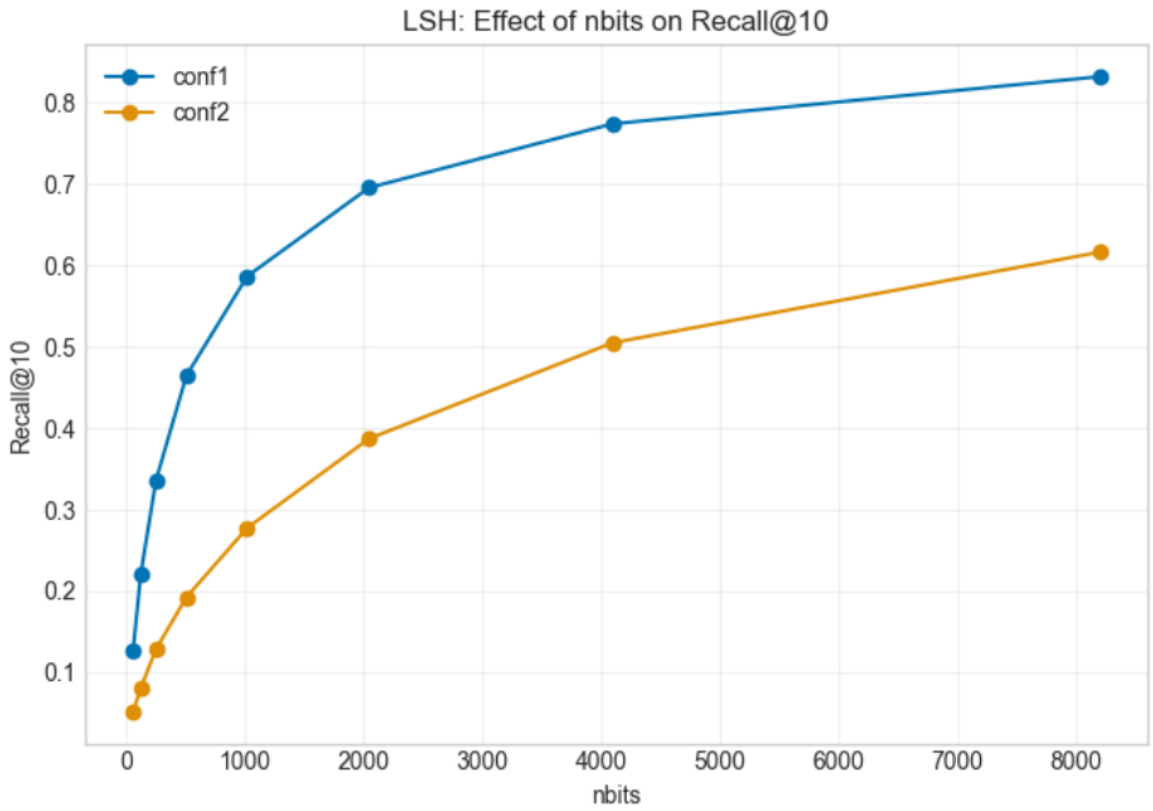


Figure 4.6: Effect of nbits on Recall@10 for LSH across Conf1 and Conf2.

Figure 4.6 shows Recall@10 as a function of nbits for both Conf1 and Conf2. Recall grows logarithmically with nbits in both cases, but Conf1 consistently outperforms Conf2 across all nbits values. At nbits=8000, Conf1 reaches approximately 0.83 while Conf2 only reaches 0.62. This suggests that the contrastively fine-tuned feature space of Conf2 is harder for LSH to approximate via random projections, likely because the more structured clustering in Conf2 requires longer hash codes to preserve similarity reliably. Neither configuration approaches the recall levels achieved by HNSW, confirming that LSH is the weaker of the two approximate methods in this setting.

Having established the results for image data, we now turn to the MovieLens experiments. The key difference here is the dimensionality: movie vectors have 69,878 dimensions compared to 2,048 for images, and the vectors are highly sparse.

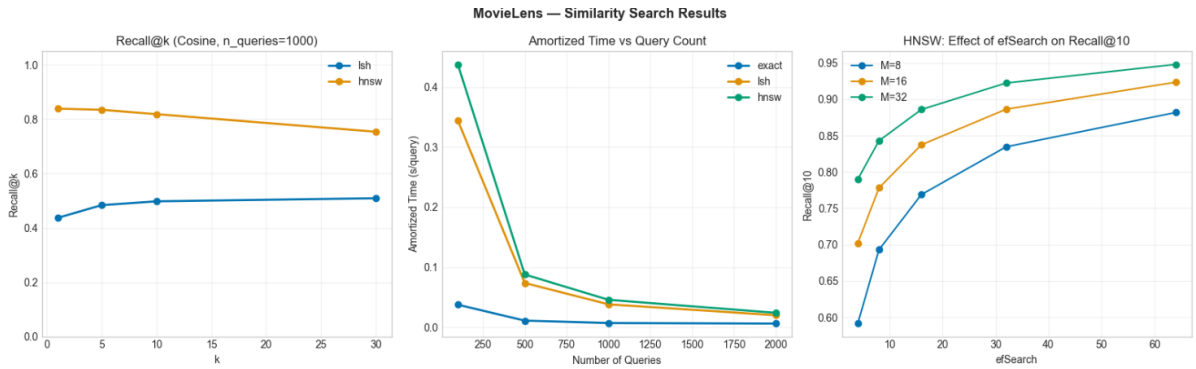


Figure 4.7: MovieLens similarity search results: Recall@k, amortized time vs query count, and HNSW efSearch effect.

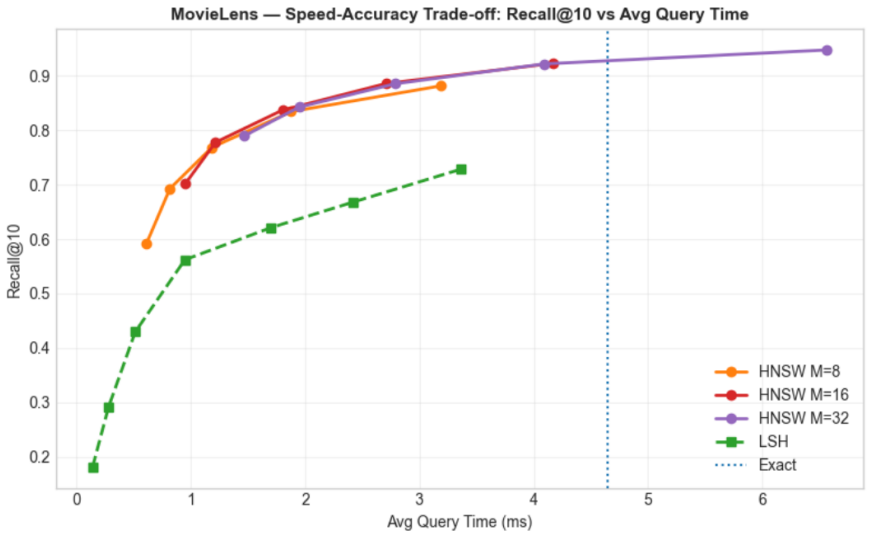


Figure 4.8: MovieLens speed-accuracy trade-off: Recall@10 vs average query time for HNSW and LSH.

Figure 4.7 shows Recall@k for HNSW and LSH at 1,000 queries using cosine distance. HNSW achieves approximately 0.83 recall at $k = 1$, declining slightly to around 0.75 at $k = 30$. LSH performs considerably worse, starting at 0.42 and reaching only 0.52 at $k = 30$. The recall gap between the two algorithms is larger here than on image data, which suggests that the high-dimensional sparse structure of movie vectors is particularly challenging for LSH's random projection mechanism.

The amortized time plot shows that all three algorithms converge towards similar query costs as the number of queries increases. At low query counts, HNSW appears expensive due to its index build time, but this cost gets amortized quickly as more queries are processed. By 2,000 queries all methods are very close in amortized time, which means HNSW's recall advantage is achieved with minimal additional amortized cost in realistic deployment scenarios.

The efSearch analysis for MovieLens mirrors the image results: higher M and higher efSearch both improve recall, with $M = 32$ reaching approximately 0.95 at efSearch=64.

Figure 4.8 further illustrates the speed-accuracy trade-off for MovieLens retrieval. HNSW consistently achieves the most favorable balance between recall and query efficiency across the evaluated configurations. Increasing M improves recall, but with diminishing returns once recall approaches near-exact levels. In contrast, LSH requires increasingly large hash sizes to improve recall, yet remains both slower and substantially less accurate than HNSW throughout the evaluated range.

4.2 Out-of-Distribution Detection Task

This section presents results for distinguishing in-distribution queries (InDQuery, ImageNet100) from out-of-distribution queries (Places365) using Deep Nearest Neighbors and NN-Guide. Both strategies are evaluated using AUC-ROC across exact kNN, HNSW, and a random baseline for $k \in \{1, 5, 10, 30\}$.

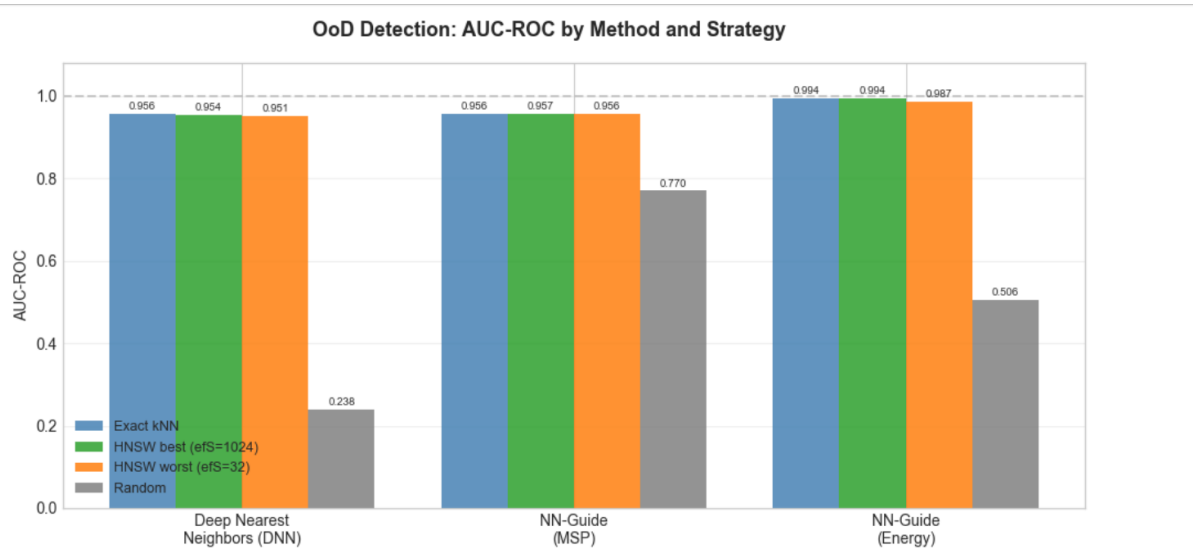


Figure 4.9: AUC-ROC achieved by each algorithm and OoD detection strategy, averaged over all values of k. Bars show Exact kNN, best HNSW (efS=1024), worst HNSW (efS=32), and Random baseline.

Figure 4.9 summarises the AUC-ROC achieved by each algorithm and strategy, averaged over all values of k. Across all evaluated OoD detection strategies, HNSW closely matches exact kNN while operating at substantially lower query times (below 1 ms per query, compared to approximately 2.7 ms for exact search; see Figure 4.10), suggesting that the approximation introduced by HNSW does not meaningfully reduce OoD detection quality.

The random baseline also highlights substantial differences between the scoring strategies. DNN collapses far below chance level when neighbour structure is destroyed, while NN-Guide remains considerably more stable, particularly when combined with MSP

Effect of the search algorithm

The random baseline reveals how much of the detection signal comes from the kNN structure rather than the scoring function alone. For Deep Nearest Neighbors, random neighbour selection yields an AUC-ROC of only 0.238 well below the 0.5 chance level. One possible explanation for this counterintuitive result is that the DNN score is based solely on the distance to a single neighbour: when that neighbour is chosen at random, OoD queries happen to receive higher scores on average than in-distribution queries, effectively inverting the ranking. For NN-Guide with Energy confidence, the random baseline sits near chance (0.506), indicating that neighbour similarity is essential and the confidence term alone cannot carry the detection. For NN-Guide with MSP, the random baseline is considerably higher (0.770), suggesting that the classification confidence $\text{conf}(q)$ already provides a meaningful OoD signal on its own, with the neighbour similarity term providing additional improvement.

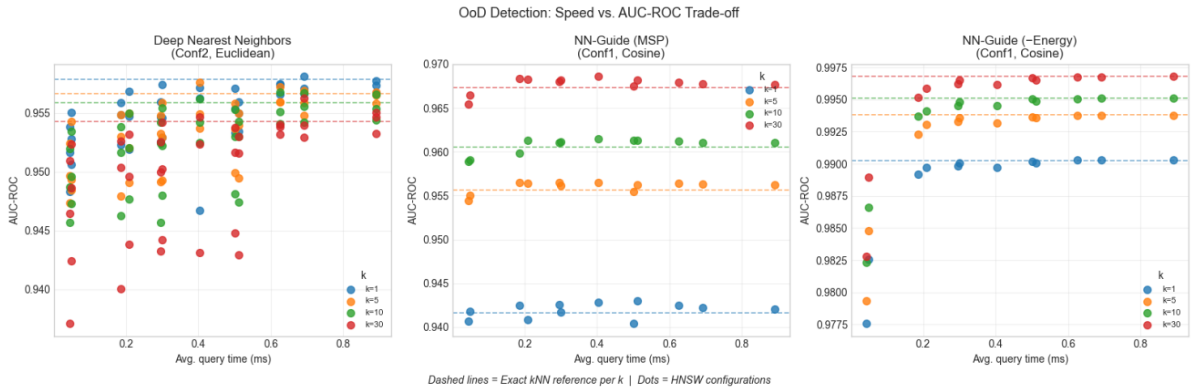


Figure 4.10: Speed–accuracy trade-off for OoD detection. Each point represents one HNSW hyperparameter configuration (M, `efSearch`, k). Dashed horizontal lines mark the Exact kNN reference for each k. Query times are derived from the same FAISS index used in Task 1.

Sensitivity to hyperparameters

Figure 4.10 shows the speed–accuracy trade-off across all HNSW configurations. The DNN scatter appears more dispersed than the two NN-Guide variants. This behaviour is consistent with the structure of the DNN scoring function: DNN uses only the distance to the single k-th nearest neighbour, meaning that individual approximation errors may directly affect the score. However, even the most dispersed DNN configurations remain relatively close to the exact kNN reference lines, indicating that the absolute reduction in detection quality remains modest

NN-Guide, by contrast, averages cosine similarities over all k neighbours, which smooths out the effect of retrieving an occasional suboptimal neighbour.

Effect of k

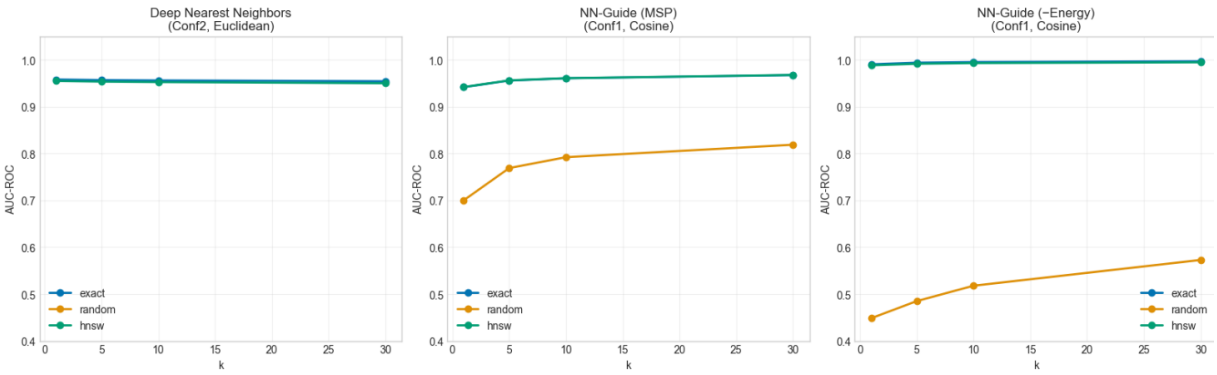


Figure 4.11: AUC-ROC as a function of k for each algorithm and strategy. Exact kNN and HNSW overlap almost completely across all strategies, suggesting that approximation quality remains stable across different values of k k.

Figure 4.11 shows AUC-ROC as a function of k for each strategy. For DNN, the curves lie very close together (AUC-ROC spanning roughly 0.950–0.960), indicating that the choice of k has little impact on detection quality. For NN-Guide, the spread is larger: with MSP confidence the AUC-ROC rises from 0.942 at $k = 1$ to 0.967 at $k = 30$, and with Energy confidence from 0.990 to 0.997. Larger k may provide more stable estimates of the confidence-weighted similarity, which benefits the NN-Guide score.

Comparison of strategies

NN-Guide with Energy confidence achieves the highest AUC-ROC overall (up to 0.997 with exact kNN at $k = 30$), followed by NN-Guide with MSP (up to 0.967) and DNN (up to 0.958). The energy score appears to provide a more informative confidence signal than MSP in this setting because it integrates over all class logits rather than relying only on the single highest probability, potentially making it more sensitive to the distributional shift between ImageNet100 and Places365.

4.3 Recommendation Task

This section presents results for item-based collaborative filtering on MovieLens 10M. Recommendation quality is measured using Hit Rate@k, evaluated across exact kNN, HNSW, LSH, and a random baseline for $k \in \{1, 5, 10, 30\}$.

Overall performance

Hit Rate@k increases with k across all algorithms, which is expected: a longer recommendation list gives a higher probability of including an item the user has actually rated. The random baseline achieves a Hit Rate near zero for all values of k, suggesting that item similarity from the kNN search is the primary driver of recommendation quality rather than the popularity of items in the dataset.

Exact kNN achieves the highest Hit Rate@k at every k, reaching approximately 0.22 at $k = 30$. All three HNSW configurations (M=8, 16, 32) with their best `efSearch` setting closely match the exact baseline almost perfectly, with differences too small to observe in Figure 4.12. LSH with its best `nbits` setting trails the other methods by a small but consistent margin across all values of k.

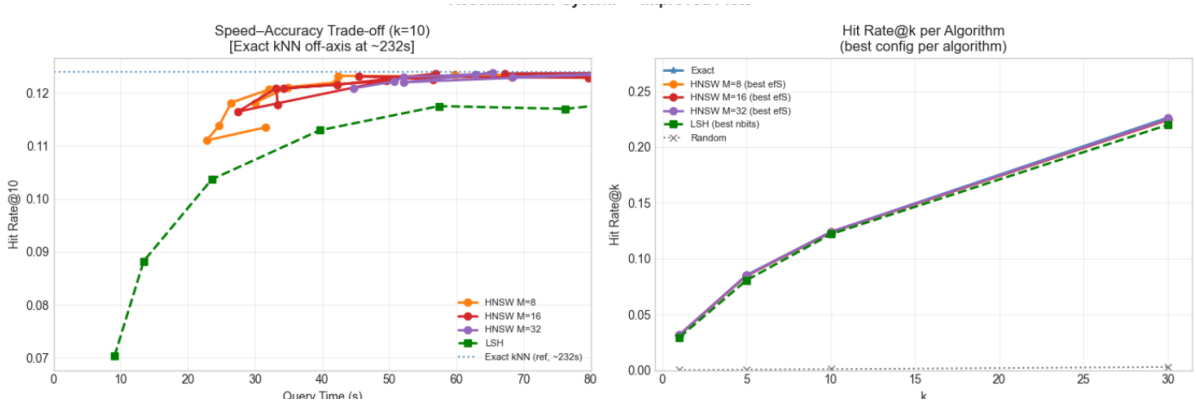


Figure 4.12: Recommender system results on MovieLens 10M. *Left:* Speed–accuracy trade-off at $k = 10$; Exact kNN lies off-axis at approximately 230 s and is shown as a dashed reference line. *Right:* Hit Rate@k for the best configuration of each algorithm.

Speed–accuracy trade-off

The left panel of Figure 4.12 shows the speed–accuracy trade-off at $k = 10$. Exact kNN requires approximately 232 seconds of total query time, while HNSW reaches near-identical Hit Rate@10 in 40–60 seconds depending on the M value. Lower M values (M=8) converge quickly at the cost of a slightly lower Hit Rate ceiling, while larger M values (M=32) require slightly more time before converging but reach the same level as exact kNN. LSH improves steadily with increasing `nbits` but does not reach the same Hit Rate

as HNSW within the evaluated range, and its query time increases more rapidly relative to the achieved improvement in Hit Rate.

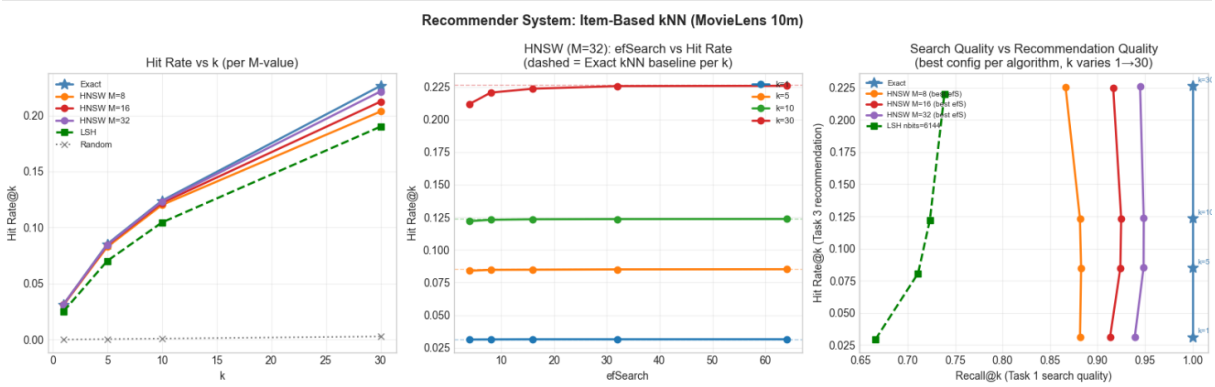


Figure 4.13: Detailed recommender system analysis. *Left:* Hit Rate@k as a function of k per algorithm. *Centre:* Effect of `efSearch` on Hit Rate@k for HNSW (M=32); dashed lines show the Exact kNN baseline per k. *Right:* Search quality (Recall@k) vs. recommendation quality (Hit Rate@k) for each algorithm configuration.

Sensitivity to hyperparameters

The centre panel of Figure 4.13 shows that Hit Rate@k is largely insensitive to `efSearch` once it exceeds approximately 16–32: the curves flatten quickly and additional search effort yields diminishing returns. This may indicate that the item neighbourhood structure in the MovieLens feature space is already sufficiently captured by moderate `efSearch` values, and there is little benefit in increasing it further at the cost of query time.

Search quality and recommendation quality

The left panel of Figure 4.13 shows that HNSW remains visually almost indistinguishable from exact kNN across all values of k, while the right panel shows a clear positive relationship between Recall@k (search quality from Task 1) and Hit Rate@k (recommendation quality): configurations that retrieve more of the true nearest neighbours also produce better recommendations. The positive relationship suggests that higher nearest-neighbour recall generally improves recommendation quality, although the effect appears limited once recall reaches moderate-to-high levels. The relationship also appears non-linear, with recommendation quality saturating once retrieval recall approaches near-exact levels. The approximation error introduced by HNSW therefore appears small enough to leave recommendation quality largely unchanged relative to exact kNN.

Chapter 5

Discussion

This chapter interprets the experimental results in light of the three hypotheses stated in Chapter 2, draws connections to the existing literature, and identifies cross-task patterns that cut across the individual evaluations.

5.1 Similarity Search: HNSW as a Near-Lossless Approximation

The results for the similarity search task strongly support H1. HNSW achieves Recall@k values close to 0.99 while delivering a query-time speedup of approximately 5× compared to exact brute-force search. The trade-off curve also flattens beyond `efSearch` = 64, suggesting diminishing returns from further graph exploration.

This behaviour is consistent with the findings reported by Malkov and Yashunin [6], where the hierarchical graph structure enables efficient navigation towards relevant neighbourhoods through logarithmic-style traversal across layers. The results therefore suggest that a relatively small candidate set is often sufficient to recover the true nearest neighbours with very high probability.

In contrast to HNSW, LSH requires substantially larger hash sizes to approach comparable recall levels. This behaviour is consistent with the known limitations of random-projection hashing in moderately high-dimensional feature spaces, where preserving neighbourhood structure becomes increasingly difficult [4]. The gap between LSH and HNSW observed in our experiments is therefore not surprising; it is consistent with the broader literature documenting that graph-based indices dominate hash-based ones on high-dimensional dense vectors [1].

Taken together, the similarity search results suggest that the approximation introduced by HNSW remains practically small at the operating points used in the remaining tasks.

5.2 Out-of-Distribution Detection: An Unexpected Sensitivity Ordering

The OoD detection results partially support H2 but contradict its key comparative prediction. HNSW and exact search yield indistinguishable AUC-ROC values for both DNN and NN-Guide across all `efSearch` settings tested, confirming the first part of H2 that a high-recall ANN index preserves OoD detection performance.

However, the prediction that NN-Guide would be more sensitive to approximation error than DNN was not supported by the experimental results. Instead, DNN exhibited greater variability in AUC-ROC as `efSearch` decreased, while NN-Guide remained comparatively stable across the evaluated settings.

One possible explanation for this outcome lies in the structural differences between the two scoring functions. DNN [9] bases its score on the distance to the *single* k-th nearest neighbour: $S_{\text{InD}}(q) = -d(q, z_k)$. A single missed neighbour can therefore shift the score substantially, making DNN inherently more sensitive to any perturbation of the retrieved candidate set. NN-Guide [7], by contrast, blends a nearest-neighbour similarity term with a softmax confidence score averaged over a neighbourhood, effectively spreading the signal across multiple retrieved examples. This averaging may partially reduce sensitivity to individual neighbour retrieval errors: even if one or two neighbours are replaced by slightly suboptimal candidates, the aggregate score changes relatively little.

The random-baseline results further support this interpretation. Random neighbour selection reduces DNN performance to approximately AUC-ROC = 0.24, which falls substantially below chance level. This suggests that DNN depends critically on meaningful neighbourhood structure and on preserving the relative ordering of distances between samples. NN-Guide Energy random reaches approximately 0.51 (near chance), while NN-Guide MSP random remains at approximately 0.77, showing that the softmax confidence signal alone carries considerable discriminative power independent of any nearest-neighbour geometry.

These findings suggest a practical design principle: scoring functions that average over a neighbourhood are inherently more robust to ANN approximation than those relying on a single extreme-order statistic.

5.3 Recommendation: Collaborative Filtering is Robust to Approximation

The recommendation experiments suggest that item-based collaborative filtering is comparatively robust to moderate approximation errors in nearest-neighbour retrieval. HNSW matches exact Hit Rate@k at moderate `efSearch` values (approximately 16 to 32), and

further increasing `efSearch` yields no measurable improvement.
Item-based collaborative filtering computes recommendations by retrieving the most similar items to those a user has previously rated, then aggregating their ratings [8]. Because recommendation systems often admit multiple similarly relevant neighbours, moderate reductions in nearest-neighbour recall do not necessarily translate into substantial losses in recommendation quality. This aggregation-based robustness is analogous to the smoothing effect observed for NN-Guide in the OoD task: both methods spread their signal over multiple neighbours rather than relying on a single extreme value.
The MovieLens 10M results therefore suggest that approximate item similarity search can serve as a practical optimisation for recommender systems under the evaluated settings, which is consistent with the practical deployment rationale motivating the original HNSW paper [6].

5.4 Cross-Task Synthesis

Taken together, the experiments suggest that the practical impact of ANN approximation depends strongly on how directly the downstream task relies on exact neighbourhood structure. HNSW consistently occupied the Pareto frontier of the recall–speed trade-off across all three tasks, while LSH required substantially more computation to reach comparable accuracy, regardless of whether the downstream metric is Recall@k, AUC-ROC, or Hit Rate@k.
A second cross-task observation concerns the role of aggregation. Tasks and scoring functions that aggregate information over a neighbourhood (NN-Guide, item-based CF) proved more robust to ANN approximation than those that rely on a single retrieved neighbour (DNN). This distinction has practical implications: when deploying ANN indices in pipelines that currently use exact search, practitioners should examine whether their downstream scoring function is a single-order-statistic or an aggregate, and calibrate the index parameters accordingly.
Finally, the contrastive fine-tuning used in Conf2 (used by DNN) versus the standard pre-trained weights of Conf1 (used by NN-Guide) introduces a confound that complicates a direct head-to-head comparison of the two OoD methods. Future work could control for the backbone by evaluating both methods under identical feature spaces, which would allow a cleaner isolation of the algorithmic contribution from the representation quality.

Chapter 6

Conclusions and Perspectives

This thesis investigated the practical impact of approximate nearest neighbor (ANN) search across three downstream tasks: similarity retrieval, out-of-distribution (OoD) detection, and item-based collaborative filtering recommendation.

The results consistently demonstrated that HNSW provides a substantially more favorable speed-accuracy trade-off than LSH across all evaluated settings. In the similarity search experiments, HNSW achieved recall values close to exact k-nearest neighbor search while reducing query time by approximately a factor of five. LSH, by contrast, required significantly larger hash sizes to approach comparable recall levels and generally remained both slower and less accurate than HNSW in high-dimensional settings. These findings support H1.

For OoD detection, the experiments demonstrated that approximate search can preserve detection quality when recall remains sufficiently high. HNSW maintained AUC-ROC values very close to exact kNN across both DNN and NN-Guide, indicating that ANN approximation does not necessarily degrade downstream reliability. However, the experiments also revealed an unexpected sensitivity ordering: DNN proved more sensitive to approximation effects than NN-Guide, contrary to the original hypothesis. This suggests that scoring functions aggregating information across multiple neighbours may be inherently more robust to ANN approximation than methods relying on a single nearest-neighbour statistic. H2 was therefore only partially supported.

In the recommendation experiments, item-based collaborative filtering proved comparatively tolerant to approximate search. HNSW achieved recommendation quality nearly identical to exact kNN at moderate `efSearch` values while substantially reducing query time. Furthermore, improvements in nearest-neighbour recall did not translate proportionally into recommendation quality gains, indicating that recommendation systems can tolerate moderate approximation error without meaningful performance degradation. These findings support H3.

Taken together, the experiments suggest that the practical acceptability of ANN search depends less on raw nearest-neighbour recall alone and more on the sensitivity character-

istics of the downstream task. Tasks relying directly on exact neighbourhood structure are more vulnerable to approximation error, while aggregation-based tasks exhibit substantially greater robustness.

Future work could extend the study by evaluating larger datasets, additional ANN methods such as IVF or Product Quantization, and a broader range of downstream applications. In the OoD setting, future experiments could also isolate the effect of feature representation by evaluating all methods using identical backbone architectures.

Overall, the results indicate that HNSW can serve as a practical replacement for exact nearest-neighbour search in many real-world systems, achieving large efficiency gains while preserving downstream performance across a diverse set of applications.

Bibliography

[1] Martin Aumüller, Erik Bernhardsson, and Alexander Faithfull. Ann-benchmarks: A benchmarking tool for approximate nearest neighbor algorithms. *Information Systems*, 87:101374, 2020.

[2] Jia Deng, Wei Dong, Richard Socher, Li-Jia Li, Kai Li, and Li Fei-Fei. Imagenet: A large-scale hierarchical image database. In *2009 IEEE Conference on Computer Vision and Pattern Recognition*, pages 248–255, 2009.

[3] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. The faiss library. *IEEE Transactions on Big Data*, 2025.

[4] Aristides Gionis, Piotr Indyk, and Rajeev Motwani. Similarity search in high dimensions via hashing. In *Proceedings of the 25th International Conference on Very Large Data Bases (VLDB)*, pages 518–529, 1999.

[5] F. Maxwell Harper and Joseph A. Konstan. The movielens datasets: History and context. *ACM Transactions on Interactive Intelligent Systems*, 5(4):1–19, 2015.

[6] Yury A Malkov and Dmitry A Yashunin. Efficient and robust approximate nearest neighbor search using hierarchical navigable small world graphs. *IEEE transactions on pattern analysis and machine intelligence*, 42(4):824–836, 2018.

[7] Jaewoo Park, Yoon Gyo Jung, and Andrew Beng Jin Teoh. Nearest neighbor guidance for out-of-distribution detection. In *Proceedings of the IEEE/CVF International Conference on Computer Vision*, pages 1686–1695, 2023.

[8] Badrul Sarwar, George Karypis, Joseph Konstan, and John Riedl. Item-based collaborative filtering recommendation algorithms. In *Proceedings of the 10th International Conference on World Wide Web*, pages 285–295, 2001.

[9] Yiyou Sun, Yifei Ming, Xiaojin Zhu, and Yixuan Li. Out-of-distribution detection with deep nearest neighbors. In *International Conference on Machine Learning*, pages 20827–20840, 2022.

[10] Sergey Zagoruyko and Nikos Komodakis. Wide residual networks. *arXiv preprint arXiv:1605.07146*, 2016.

[11] Bolei Zhou, Agata Lapedriza, Aditya Khosla, Aude Oliva, and Antonio Torralba. Places: A 10 million image database for scene recognition. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 40(6):1452–1464, 2017.

Appendix: Individual Report

Task Division Not Applicable (individual project).

Name and Individual Contribution Not Applicable.

Organizational Issues One organizational challenge during the project was balancing the workload with parallel university courses and overlapping project deadlines. In addition, several experiments required substantial processing time for feature extraction, ANN index construction, and repeated evaluation across multiple hyperparameter configurations

Went Better than Planned The overall experimental workflow progressed more smoothly than expected. Most configurations produced stable and usable results with relatively few major issues during evaluation and analysis

Went Worse than Planned Managing experiment runtime and preprocessing time proved more challenging than expected. In particular, forgetting to extract one dataset early in the process resulted in an additional 7–8 hours of processing time before a usable result could be produced

Main Technical Takeaways A major technical takeaway from the project was learning how ANN retrieval quality interacts with downstream machine learning performance rather than relying only on retrieval metric. The project also provided practical experience with large-scale experimentation, feature extraction pipelines, hyperparameter evaluation, and performance trade-off analysis in machine learning systems. In particular, the results demonstrated that different applications vary significantly in their sensitivity to ANN approximation errors, while HNSW consistently achieved the most favorable speed-accuracy trade-off across the evaluated settings.

Other Takeaways The project highlighted the importance of structured experiment management, clear result visualization, and maintaining a consistent workflow throughout both implementation and report writing. It also improved my ability to plan and manage long-running independent projects

What If... If the project were to be repeated, I would spend more time planning the experimental workflow in advance, particularly regarding dataset preprocessing, feature extraction, and expected runtime for large experiments. Establishing a clearer execution plan and runtime estimates earlier in the project would likely have improved time management and reduced unnecessary reruns. I would also make greater use of supervision and related work earlier in the process to compare implementation strategies and experimental setups.